16

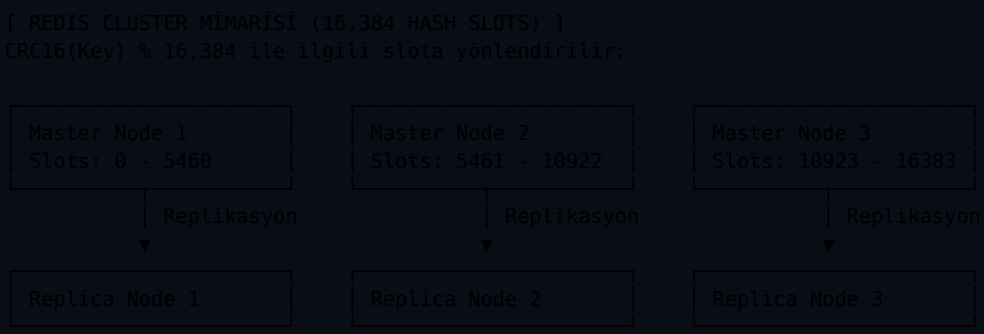
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Dağıtık Önbellek: Redis ve Memcached Mimarisi (Sentinel vs Cluster)

Distributed Cache: Redis vs Memcached (Sentinel & Cluster Sharding)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Redis ile Memcached iki ünlü yarış arabası gibidir: Memcached sadece dümdüz ileri giden, çok hafif ve basit bir drag yarışçısıdır (Sadece Key-Value tutar, çok çekirdek kullanır). Redis ise İsviçre çakısı gibi donatılmış bir Formula 1 aracıdır! Listeler, Kümeler (Sets), Sıralı Kümeler (Sorted Sets), Bitmap'ler, Pub/Sub ve diske kaydetme (Persistence) yetenekleri vardır. Redis Cluster ise bu Formula 1 filosunun 3 farklı pistte (16,384 yuva - hash slot) yarışarak yükü paylaşmasıdır!

🇬🇧 ENGLISH (ELI5)

Redis and Memcached are two performance sports cars: Memcached is a stripped-down dragster built for raw multi-threaded speed (pure Key-Value strings). Redis is a hypercar engineered with an entire Swiss Army knife! It natively manipulates Hashes, Lists, Sets, Sorted Sets (ZSET for leaderboards), Bitmaps, HyperLogLogs, and Pub/Sub streams with disk durability (RDB/AOF). Redis Cluster partitions keys across 16,384 distributed hash slots for horizontal scaling!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Memcached: Multi-threaded mimariye sahiptir; çok çekirdekli makinelerin CPU gücünü tam kullanır; veri tipleri yoktur (sadece binary string); disk kalıcılığı yoktur. Redis: Çekirdek motoru tek iş parçacıklıdır (Single-threaded event loop), bu sayede race condition ve kilit (lock) kargaşası olmadan komutları atomik yürütür (Network IO ise multithreaded'dir). Redis Mimarileri: 1) Redis Sentinel: Yüksek erişilebilirlik (HA) sağlar; Master çökerse otomatik Replica'yı Master yapar (Failover). 2) Redis Cluster: Yatay ölçekleme (Sharding) sağlar. Anahtarlar `CRC16(key) % 16384` formülüyle 16,384 Hash Slot'a bölünür ve master düğümler arasında paylaştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Memcached utilizes a multithreaded architecture maximizing multi-core symmetric multiprocessing, but supports only basic string key-values without disk durability. Redis runs an ultra-fast non-blocking single-threaded event loop, guaranteeing atomic executions across rich data structures (Lists, ZSETs, Streams, Hashes) without mutex locking overhead. Scalability options: 1) Redis Sentinel provides automatic master-replica failover monitoring. 2) Redis Cluster delivers true horizontal sharding: keys map to 16,384 hash slots via `CRC16(key) % 16384`, distributing partitions across autonomous master-replica shards with zero single points of failure.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Redis Hash Slot Hesaplayıcı Simülasyonu (CRC16 % 16384)
public class RedisClusterSlotRouter {
    private static final int TOTAL_SLOTS = 16384;

    // Basitleştirilmiş Hash Slot Hesaplama
    public static int calculateSlot(String key) {
        // Redis Hash Tag Desteği: {user:101}.orders -> Sadece {} içi hashlenir!
        int start = key.indexOf('{');
        int end = key.indexOf('}');
        String hashKey = (start != -1 && end > start) ? key.substring(start + 1, end) : key;

        return Math.abs(hashKey.hashCode() % TOTAL_SLOTS);
    }
}

void main() {
    String key1 = "product:iphone16";
    String key2 = "user:9941";
    String key3 = "{tenant:acme}.orders";
    String key4 = "{tenant:acme}.invoices"; // key3 ile KESİNLİKLE aynı slot'a gider!

    IO.println(key1 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key1));
    IO.println(key2 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key2));
    IO.println(key3 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key3));
    IO.println(key4 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key4));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Redis Cluster'da çoklu anahtar (MGET, MSET veya Transactions) çalıştırırken anahtarlar farklı slotlardaysa 'CROSSSLOT Keys in request don't hash to the same slot' hatası patlar! İlişkili verilerin aynı slota düşmesini sağlamak için Hash Tag `\{tenant_id\}.key` sözdizimini kullanın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Redis Cluster throws CROSSSLOT errors when batching multi-key commands across different hash slots. Enforce colocation using Hash Tags (`\{user_id\}.profile` and `\{user_id\}.orders`) so related keys evaluate to the exact same shard.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis zengin veri tipleri ve atomik işlemleriyle modern sistemlerin vazgeçilmezidir.
- Redis Cluster, 16,384 Hash Slot yapısıyla veriyi onlarca sunucuya şeffafça böler.
- Hash Tags (\{tag\}) tek işlemde birden fazla anahtarı sorgulamak için slot sabitleme sağlar.

🎯 Key Takeaways for the Interview

- Redis is the premier caching engine due to its rich native data types and atomic primitives.
- Redis Cluster transparently shards datasets across 16,384 deterministic hash slots.
- Hash Tags ensure co-partitioning for multi-key transactional pipelines on clustered shards.